

plot_profil_saccade

July 24, 2026

0.1 Code de génération des profils de vitesse saccadés

Profils de vitesse réels avec saccade de désalignement — accouplement fourche-T, modèle exact, périodique de 180°.

Modèle : un point (ergot) est entraîné à rayon fixe R autour de l'axe moteur. L'axe tige, décalé radialement de e , “regarde” ce même point. Le rapport de transmission instantané exact, sur un DEMI-tour, est :

$$\frac{\omega_2}{\omega_1} = \frac{R(R + e \cos \alpha)}{R^2 + e^2 + 2Re \cos \alpha}$$

La fourche en T (tige) étant symétrique (deux bras à 180°), ce rapport se répète à l'identique tous les 180° de rotation moteur (et non tous les 360°) : le second bras reproduit exactement le même relais que le premier. On observe donc une discontinuité de vitesse (mais pas de position) à chaque demi-tour, lorsque le bras actif change. Aucune approximation n'est faite (pas de linéarisation, pas d'hypothèse de petit angle).

Deux profils sont simulés séparément : - **Remontage** (mode réserve de marche, Mvt01) : 50 tr/min, 34,3 tr (N_{tige}), accélération réelle du système limitée à 5000 tr/min² (cf. section Dimensionnement). - **Vieillessement accéléré** : 350 tr/min visés, 2 tr — à l'accélération réelle de 5000 tr/min², la distance est insuffisante pour atteindre 350 tr/min (profil triangulaire, pic 100 tr/min seulement). Pour illustrer la forme trapézoïdale visée, ce graphe utilise donc une accélération illustrative de 100 000 tr/min², suffisante pour atteindre le palier à 350 tr/min sur les 2 tr disponibles ; ce n'est pas la valeur réelle du système (cf. §6.4).

Paramètres géométriques : - $e = 2,58$ mm (Δh , désalignement radial réel maximal entre les 6 mouvements documentés) - $R = 12,5$ mm (rayon d'entraînement, demi-écart de la fourche)

```
[1]: import numpy as np
import matplotlib.pyplot as plt

BLUE = "#378ADD"
ORANGE = "#D85A30"

# Parametres geometriques
R = 12.5          # rayon d'entraînement (mm)
E = 2.58         # desalignement radial reel = Delta h (mm)
N = 4000         # points de simulation
```

```

def profil_trapeze(v_max, distance, accel, N=N):
    """
    Profil de vitesse trapezoidal (ou triangulaire si la distance est trop
    courte pour atteindre v_max) a acceleration/deceleration constante.
    Retourne t (s), v (tr/min).
    """
    t_acc = v_max / accel #_
    ↪ min
    d_acc = v_max**2 / (2 * accel) # tr
    if 2 * d_acc <= distance:
        d_cruise = distance - 2 * d_acc
        t_cruise = d_cruise / v_max
        t1, t2 = t_acc, t_acc + t_cruise
        v_pic = v_max
    else:
        v_pic = np.sqrt(accel * distance) #_
    ↪ profil triangulaire
    t_acc = v_pic / accel
    t1 = t2 = t_acc
    t_total = t2 + t_acc if 2 * d_acc <= distance else 2 * t_acc
    t = np.linspace(0, t_total, N) # min
    v = np.where(t <= t1, accel * t,
        np.where(t <= t2, v_pic, v_pic - accel * (t - t2)))
    return t * 60, v #_
    ↪ (s, tr/min)

def with_jitter(t_s, v_ideal, e, R):
    """Vitesse avec saccade periodique de 180 deg (symetrie de la fourche en T).
    ↪ """
    omega_rad = v_ideal * 2 * np.pi / 60
    dt = t_s[1] - t_s[0]
    alpha = np.cumsum(omega_rad) * dt
    alpha_mod = np.mod(alpha, np.pi)
    ratio = R * (R + e * np.cos(alpha_mod)) / (R**2 + e**2 + 2 * R * e * np.
    ↪ cos(alpha_mod))
    return v_ideal * ratio

def eps_bounds(e, R):
    """Bornes exactes de l'erreur de vitesse relative (min en alpha=0, max en_
    ↪ alpha=pi)."""
    return -e / (R + e) * 100, e / (R - e) * 100

eps_min, eps_max = eps_bounds(E, R)

print("=" * 55)
print(" Accouplement fourche-T -- analyse de desalignement (exact)")

```

```

print("=" * 55)
print(f" Rayon d'entrainement      R      = {R:.1f} mm")
print(f" Desalignement reel       e      = {E:.2f} mm (Delta h)")
print(f" Erreur de vitesse         min/max = {eps_min:.1f}% / {eps_max:.1f}%")
print("=" * 55)

```

```

=====
Accouplement fourche-T -- analyse de desalignement (exact)
=====

```

```

Rayon d'entrainement      R      = 12.5 mm
Desalignement reel       e      = 2.58 mm (Delta h)
Erreur de vitesse        min/max = -17.1% / 26.0%
=====

```

```

/home/colin/.local/lib/python3.10/site-
packages/matplotlib/projections/_init__.py:63: UserWarning: Unable to import
Axes3D. This may be due to multiple versions of Matplotlib being installed (e.g.
as a system package and as a pip package). As a result, the 3D projection is not
available.

```

```

warnings.warn("Unable to import Axes3D. This may be due to multiple versions
of "

```

```

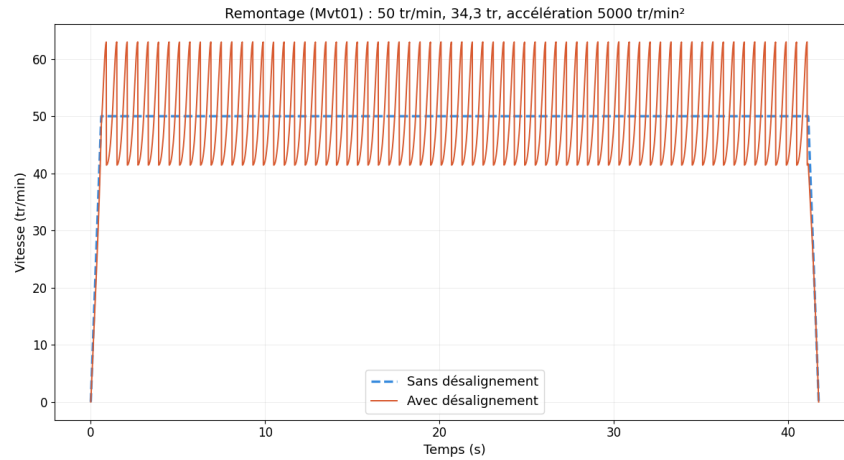
[2]: # Figure 6.19a : remontage (Mvt01), 50 tr/min, 34,3 tr, accel reelle 5000 tr/
    ↪ min2
ACCEL_REEL = 5000.0 # tr/min2, valeur reelle du systeme (cahier des charges)
t1_s, v1 = profil_trapeze(v_max=50.0, distance=34.30, accel=ACCEL_REEL)
actual1 = with_jitter(t1_s, v1, E, R)

fig, ax1 = plt.subplots(figsize=(12, 7.5))
fig.suptitle(f"Profil de vitesse réel en mode réserve de marche (Mvt01), "
             f"avec saccade périodique due au désalignement $e=\\Delta h={E:.2f}$ mm",
             ↪ 2f}$ mm",
             fontsize=16, fontweight="bold")
ax1.plot(t1_s, v1, color=BLUE, lw=2.4, ls="--", label="Sans désalignement")
ax1.plot(t1_s, actual1, color=ORANGE, lw=1.4, label="Avec désalignement")
ax1.set_xlabel("Temps (s)", fontsize=13)
ax1.set_ylabel("Vitesse (tr/min)", fontsize=13)
ax1.set_title("Remontage (Mvt01) : 50 tr/min, 34,3 tr, accélération 5000 tr/
    ↪ min2", fontsize=14)
ax1.tick_params(labels=12)
ax1.legend(fontsize=13, loc="lower center")
ax1.grid(True, lw=0.4, alpha=0.4)

fig.tight_layout(rect=[0, 0, 1, 0.93])
fig.savefig("../figures/Implementation/fig_profil_remontage.png", dpi=150,
    ↪ bbox_inches="tight")
plt.show()

```

Profil de vitesse réel en mode réserve de marche (Mvt01), avec saccade périodique due au désalignement $e = \Delta h = 2.58 \text{ mm}$



```
[3]: # Figure 6.19b : vieillissement, 350 tr/min visés, 2 tr
# Accel réelle (5000 tr/min²) insuffisante pour atteindre 350 tr/min sur 2 tr :
# profil triangulaire, pic reel < 350 tr/min. Pour illustrer la forme
# trapezoidale visée, on utilise ici une accélération illustrative de
# 100 000 tr/min² (non representative du système reel, cf. §6.4).
t_tri_s, v_tri = profil_trapeze(v_max=350.0, distance=2.0, accel=ACCEL_REEL)
print(f" Vieillissement (accel réelle {ACCEL_REEL:.0f} tr/min²) : distance_
↳insuffisante ;")
print(f" profil triangulaire, pic reel = {v_tri.max():.1f} tr/min")

ACCEL_ILLUSTRATIF = 100_000.0 # tr/min², illustratif uniquement (cf. §6.4)
t2_s, v2 = profil_trapeze(v_max=350.0, distance=2.0, accel=ACCEL_ILLUSTRATIF)
actual2 = with_jitter(t2_s, v2, E, R)

fig, ax2 = plt.subplots(figsize=(12, 7.5))
fig.suptitle(f"Profil de vitesse illustratif en mode vieillissement accéléré "
f"(accélération {ACCEL_ILLUSTRATIF:.0f} tr/min², non_
↳représentative du système réel),\n"
f"avec saccade périodique due au désalignement $e=\\Delta h={E:.
↳2f}$ mm",
fontSize=15, fontweight="bold")
ax2.plot(t2_s, v2, color=BLUE, lw=2.4, ls="--", label="Sans désalignement")
ax2.plot(t2_s, actual2, color=ORANGE, lw=1.8, label="Avec désalignement")
ax2.set_xlabel("Temps (s)", fontsize=13)
ax2.set_ylabel("Vitesse (tr/min)", fontsize=13)
ax2.set_title("Vieillissement : 350 tr/min visés, 2 tr (profil trapézoïdal_
↳illustratif)", fontsize=14)
ax2.tick_params(labelsize=12)
ax2.legend(fontsize=13, loc="lower center")
```

```
ax2.grid(True, lw=0.4, alpha=0.4)

fig.tight_layout(rect=[0, 0, 1, 0.90])
fig.savefig("../figures/Implementation/fig_profil_vieillessement.png",
            dpi=150, bbox_inches="tight")
plt.show()
```

Vieillessement (accél. réelle 5000 tr/min²) : distance insuffisante ;
 profil triangulaire, pic réel = 100.0 tr/min

**Profil de vitesse illustratif en mode vieillissement accéléré (accélération 100000 tr/min², non représentative du système réel),
 avec saccade périodique due au désalignement $e = \Delta h = 2.58$ mm**

